Text Emotion Recognition: An In-Depth Analysis

Ng Woon Yee
Nanyang Technological University
Singapore
U2120622C

Lye Jin Kai
Nanyang Technological University
Singapore
U2120587D

Won Tian Cong, Adriel
Nanyang Technological University
Singapore
U2122905K

***Abstract*—This paper presents a comprehensive comparative analysis of various deep learning approaches for text emotion recognition. We evaluate Attention-based Bidirectional Long Short-Term Memory (BiLSTM), Convolutional Neural Network with Bidirectional Gated Recurrent Unit (CNN-BiGRU), and various Transformer-based models including Multi-Head Attention (MHA), Multi-Query Attention (MQA), Grouped-Query Attention (GQA), and Multi-Head Latent Attention (MLA). Our experiments demonstrate that the CNN-BiGRU architecture achieves superior performance with an F1 score of 0.8855, outperforming both the Attention-based BiLSTM (0.8657) and Transformer-based models, with the best Transformer variant (GQA) reaching an F1 score of 0.8649. We analyze the computational efficiency and parameter requirements of each approach, providing insights into the trade-offs between performance and resource utilization in text emotion recognition tasks. These findings contribute to the ongoing development of more accurate and efficient emotion recognition systems with potential applications across multiple domains.recognition tasks.**

***Index Terms*—text emotion recognition, deep learning, BiLSTM, CNN-BiGRU, transformer models, attention mechanisms**

I. Introduction

Text emotion recognition refers to the task of identifying and interpreting the emotional tone conveyed in human natural language. By analyzing a text, the model predicts emotions such as happiness, sadness, anger and surprise based on the words, phrases and context used. Traditional approaches to text emotion recognition include rule-based systems and machine learning classifiers, which relied heavily on explicit feature selection and struggled to capture the complex relationships between words and emotions.

Recent advancements in deep learning have enhanced text emotion recognition by enabling models to capture both local and global features. Local features include emotions expressed at word level, while global features refer to the emotions conveyed through sentences or document level. Deep learning architectures like Bidirectional Long Short-Term Memory (BiLSTM), Convolutional Neural Network with Bidirectional Gated Recurrent Unit (CNN-BiGRU) and Transformers enhance text emotion recognition by learning these complex relationships directly from the data.

In this project, we explore and evaluate the application of these deep learning models, aiming to improve the accuracy and depth of emotion recognition in text.

II. Literature Review

A. Word Embedding (GloVe over One-Hot Encoding)

One-Hot Encoding represents each word as a binary vector where only one element is '1' and the rest are '0'. Although simple, it has limitations: it doesn't capture relationships between words with similar meanings and creates high-dimensional vectors for large vocabularies, increasing complexity. For example, in "the cat and the dog": the = [1, 0, 0, 0] cat = [0, 1, 0, 0] and = [0, 0, 1, 0] dog = [0, 0, 0, 1] In one-hot encoding, similar words like "cat" and "dog" have completely independent representations. The vector dimension equals vocabulary size, potentially reaching thousands of dimensions. Conversely, GloVe (Global Vectors for Word Representation) [2] captures semantic relationships by representing words in continuous vector spaces based on context. Words with similar meanings have similar vector representations, improving emotion recognition capabilities by detecting words with similar emotional connotations. Using the same example with 5-dimensional GloVe vectors: the = [0.10, -0.43, 0.61, 0.12, 0.38] cat = [0.53, 0.12, -0.22, 0.61, 0.02] and = [0.12, 0.51, -0.28, 0.55, -0.32] dog = [0.32, 0.11, -0.26, 0.56, 0.21] GloVe encodes semantic relationships in the vector values. Similar concepts (like "cat" and "dog") appear closer in vector space. Figure 1 illustrates vector relationships between gendered words. The vector difference "man"-"woman" represents gender in vector space. This concept extends to other word pairs: "king"-"man"+"woman"="queen"

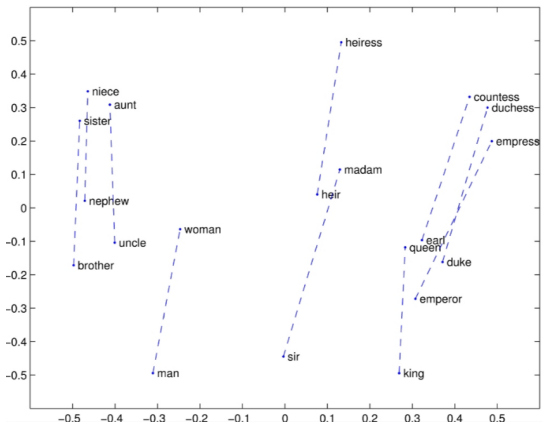

Fig. 1. Vector representation of words in GloVe embedding showing relationships between male and female words [1]

[2] . Similar patterns work for concepts like countries/capitals and comparative adjectives. While One-Hot encoding is simpler, GloVe's deeper representations of semantic relationships make it superior for emotion analysis tasks.

B. Attention-based Bidirectional Long Short-Term Memory (BiLSTM)

Long Short Term Memory (LSTM) networks are a type of Recurrent Neural Network (RNN) that are able to capture long-range dependencies in dequential data like text. Traditional RNNs face a limitation with vanishing gradients, which makes it difficult to learn long-term dependencies. By introducing gates to manage the flow of information and retain the memory of past inputs, LSTMs are able to overcome this limitation [3].

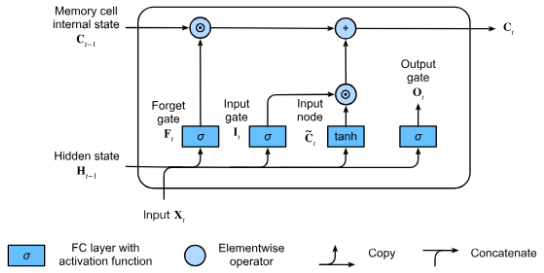

Fig. 2. Structure of LSTM cell showing forget gate, input gate, and output gate. [6]

The LSTM has three main gates: forget gate, input gate and output gate. The gating mechanism interacts with cell state (long-term memory) and hidden state (short-term memory). Previous hidden state H_{t-1} and input X are concatenated for the gates. Forget gate selects relevant information using a 0-1 vector. Input gate regulates new information entering the cell state, while candidate memory normalizes data using hyperbolic tangent activation. The output gate determines the next hidden state (H_t) for each timestep [5].

Bidirectional LSTMs (BiLSTM) improve upon LSTMs by processing sequences in both forward and backward directions, accessing context from past and future. This enhances emotion interpretation dependent on surrounding words, making BiLSTMs effective for text emotion recognition. BiLSTM

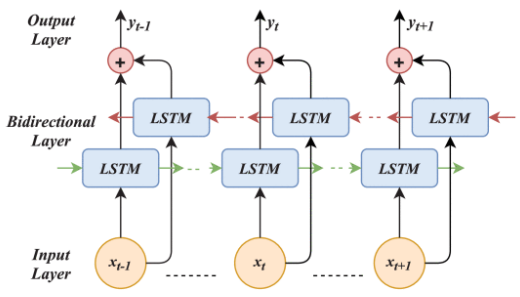

Fig. 3. Structure of BiLSTM showing forward and backward LSTM layers. [15]

combines forward and backward LSTM layers, each processing the sequence in their direction while maintaining separate hidden states. The layers concatenate after computing individual hidden states, providing context for both preceding and succeeding words. [7] Attention mechanisms enhance BiLSTM by focusing on important words in a sequence. Inspired by human information processing, attention assigns different weights to input parts, emphasizing words relevant for emotion recognition for more accurate interpretation. Attention works in three processes: reading input sequences into vector embeddings, determining dependencies by calculating attention scores, and applying attention weights to emphasize important parts of the input data. Attention layers take three inputs: query (information a token seeks), keys (information in each input token), and values (returning data after determining relevant keys). Different attention mechanisms include Additive Attention, Dot-Product Attention, Scaled Dot-Product Attention, and Self-Attention [22]. BiLSTM commonly uses Additive attention, which calculates the sum of query and key representations through a small feedforward network, producing alignment scores passed through tanh activation, then calculating attention scores with softmax for the context vector.

C. Convolutional Neural Network with Bidirectional Gated Recurrent Unit (CNN-BiGRU)

Convolutional Neural Networks (CNNs) identify local patterns within data, ideal for processing text to capture word sequences like n-grams. For text emotion recognition, CNNs detect local features representing emotion indicators, such as words suggesting happiness or sadness. CNNs apply filters across text to learn patterns at various granularity levels [14].

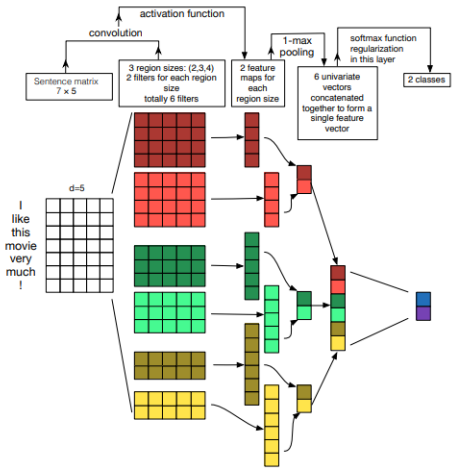

Fig. 4. Convolutional Neural Network for text processing. [4]

Though known for image processing, CNNs work well for NLP by capturing and extracting features from text data, focusing on local features compared to RNNs which capture sequential dependencies. The embedding layer comes first in CNN models, with Word2Vec and GloVe replacing earlier one-hot encoding approaches. The convolution layer maps word

embeddings as a matrix where filters slide over to capture local patterns. This produces feature maps through element-wise multiplication between filters and input. Multiple feature maps correspond to multiple filters used. Feature maps pass through an activation function (commonly ReLU) to introduce non-linearity and mitigate vanishing gradients. The pooling layer, typically max pooling, down-samples feature maps by extracting important features. The fully connected layer combines extracted features for final prediction using softmax for multi-class classification. Gated Recurrent Units (GRUs [18]) are simplified LSTMs that efficiently capture temporal dependencies in sequential data through gating mechanisms that selectively retain important information. GRUs use two

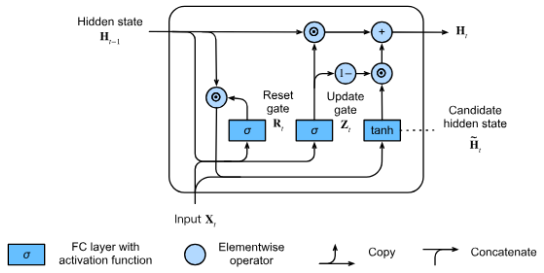

Fig. 5. Structure of Gated Recurrent Unit (GRU). [18]

primary gates: reset gate and update gate (which merges LSTM's forget and input gates). GRUs receive input X_t and previous hidden stateH_{t-1} at each timestep. The update gate controls information retention from previous states for long-term dependencies, while the reset gate controls information to forget for short-term dependencies. The candidate hidden state controls potential new memory using hyperbolic tangent activation on the concatenation of X_t and H_{t-1} via the reset gate. The final hidden state H_t is updated using the update gate to control retention between candidate and previous states.

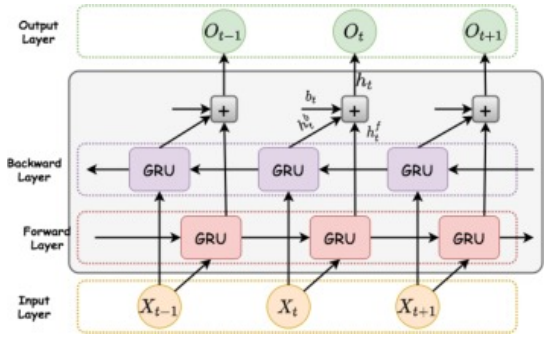

Fig. 6. Bidirectional GRU structure showing forward and backward processing. [13]

Bidirectional GRUs (BiGRUs) process information in both forward and backward directions, considering past and future context for emotion interpretation. Forward and backward layers concatenate after computing individual hidden states, providing context for both preceding and succeeding words.

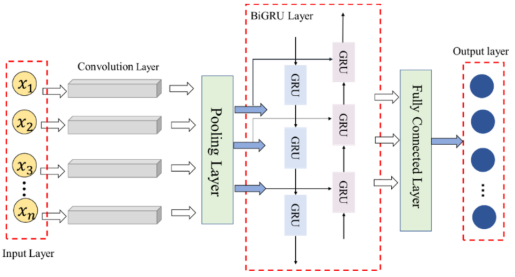

Fig. 7. CNN-BiGRU architecture combining local feature extraction with sequential processing. [6]

CNN-BiGRU combinations capture both local and global patterns: CNNs extract local patterns like emotional phrases, while BiGRUs capture broader dependencies. CNN layers follow traditional CNN flow, but pooling layer output feeds into BiGRU layers instead of fully connected layers, combining local feature extraction with broader emotional context understanding.

D. Transformer-based Models (MHA, MQA, GQA, MLA)

The paper "Attention is all you need" revolutionized Natural Language Processing in 2017. While RNNs [25], CNNs [24] and the later LSTM [23] and GRU [18] did improve the field significantly, researchers eventually encountered computational bottlenecks with these architectures. For example, RNNs and its variants like LSTM and GRU used to be the default model of choice for learning variable-length representations. However, their sequential computation inhibits parallelization. Additionally, no explicit modeling of long- and short-range dependencies have been implemented. On the other hand, although CNN offers per-layer parallelism, and could model local dependencies effectively, it requires many layers to model long-distance dependencies. Hence, attention between encoder and decoder that solve these limitations is crucial in Natural Language Processing.

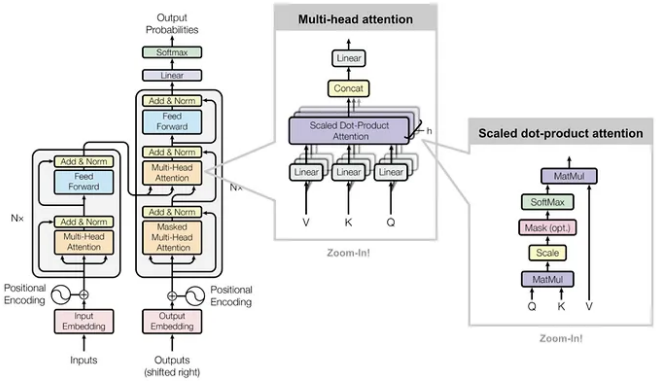

Fig. 8. Transformer Architecture. Adapted from SC4001 Lecture Note.

The transformer architecture addressed these limitations through a self-attention mechanism, which enables direct modeling of relationships between all positions in a sequence

regardless of their distance from each other. It is easy to parallelize and can replace sequential computation entirely. As there might be multiple patterns of features that we would like a model to capture, multi-head self-attention with independent learnable parameters is used in the transformer block. Besides, additional features like positional encoding, feed-forward networks (multi-layer perceptron – MLP) for non-linearity, and layer normalization to stabilize the learning are also introduced across the network.

When transformer blocks are stacked together, we first realize that MLPs usually account for a significant portion of the activations during training [28]. However, as the number of input tokens increases, the KV matrices become exponentially larger without proper KV caching, which means KV matrices might surpass the number of parameters for MLPs when the input token size is large. This problem has been discussed and attempts to solve it have been made by multiple parties. For example, Google Research first proposed KV caching in multi-head attention (MHA) [3]. Subsequently, Google also proposed multi-query attention (MQA) [26] where each attention head uses the same KV cache to reduce the activation size, but this didn't achieve good performance because there is less specialization. Then Meta came up with group-query attention (GQA) [27] where they allow more specialization by grouping attention heads together to use the same KV cache while allowing more than one KV cache to be present in the same layer – a trade-off between MHA and MQA. More recently, Multi-Head Latent Attention (MLA) [29] has been proposed to compress attention computation into a lower-dimensional latent space, offering another approach to reduce computational complexity. In this project, not only will we examine how effective transformer encoder blocks are in text emotion classification, but we will also implement different types of Transformers with various attention mechanisms to deeply investigate their differences, pros, and cons.

III. Description of Methods Used

For this experiment, GloVe embedding with 6 billion tokens and 100 dimensional vectors is used. The load_glove_embeddings() function will load the GloVe embeddings and return a dictionary of embeddings in a vector.

```
def load_glove_embeddings() -> dict:
    """
    Load GloVe embeddings
    """
    glove_dict = {}
    word_embedding_glove = load_dataset("SLU-CSCI4750/glove.6B.100d.txt")
    word_embedding_glove = word_embedding_glove["train"]

    for example in word_embedding_glove:
        split_line = example["text"].strip().split()
        word = split_line[0]
        vector = np.array(split_line[1:], dtype="float32")
        glove_dict[word] = vector

    print(f"Total GloVe words loaded: {len(glove_dict)}")
    return glove_dict
```

Fig. 9. Code snippet showing GloVe embeddings loading function

First a dictionary `glove_dict` is created for the storage of the words vectors after it has been loaded from the GloVe file. Afterwards the pretrained GloVe 6B 100d txt file is imported from the datasets library. In the for loop, the function iterates over each line for the GloVe embeddings and splits each line into words and their respective embedding vectors. `glove_dict[word]` will be the key to each corresponding vectors. Finally, the length of the print, the number of words in the dictionary and `glove_dict` is returned for the next process.

```
def create_embedding_matrix(vocab, save=False) -> dict:
    """
    Create embedding matrix for the vocabulary, include PAD and UNK too
    """
    glove_dict = load_glove_embeddings()
    vocab = vocab.union({'<PAD>', '<UNK>'})
    embedding_matrix = np.zeros((len(vocab), EMBEDDING_DIM))

    missing_words = []
    for word in vocab:
        if word not in glove_dict and word not in {'<PAD>', '<UNK>'}:
            missing_words.append(word)
    missing_words_embedding = np.mean(list(glove_dict.values()), axis=0)

    special_tokens = ['<PAD>', '<UNK>']
    regular_words = sorted([word for word in vocab if word not in special_tokens])
    all_words = special_tokens + regular_words
    word2idx = {word: idx for idx, word in enumerate(all_words)}
    for word, idx in word2idx.items():
        if word == '<PAD>':
            embedding_matrix[idx] = np.zeros(EMBEDDING_DIM)
        elif word == '<UNK>' or word in missing_words:
            embedding_matrix[idx] = missing_words_embedding
        else:
            embedding_matrix[idx] = glove_dict[word]

    if save:
        np.save(EMBEDDING_PATH, embedding_matrix)
        print(f"Embedding Matrix saved to {EMBEDDING_PATH}")

        with open(WORD2IDX_PATH, "w", encoding="utf-8") as f:
            json.dump(word2idx, f, ensure_ascii=False, indent=4)
        print(f"Word to Index mapping saved to {WORD2IDX_PATH}")

    return embedding_matrix, word2idx
```

Fig. 10. Code creating the embedding matrix

This `create_embedding_matrix` function will call the `load_glove_embedding` function to get the dictionary of words. `<PAD>` is added to handle the padding of sequences and `<UNK>` is added to represent the out of vocabulary words. Afterwards the embedding matrix is created and initialised with all zeros. The rows will be the number of vocabulary words and the columns will be 100 which is the dimension of this GloVe embedding. The `missing_words[]` will record all the words that are not in the pre-trained GloVe embedding and will be added to the list of missing words in the for loop. After the loop has ended, the default embedding value will be calculated for the unknown words using the average of all GloVe vectors. Next the special tokens(unknown words) and regular words are added and mapped to the `word2idx` dictionary. The next for loop will populate the embedding matrix with the GloVe vectors and special tokens. Each unique infex will be mapped to a word. Then the embedding matrix would be saved to the filepath so that we would no need to create the embedding matrix every time. Afterwards the

`embedding_matrix` and `word2idx` is returned.

A. Model Architectures

Three types of models used are BiLSTM with attention, CNN_BiGRU and Transformer based model. BiLSTM with Attention was used as BiLSTM is strong at capturing the past and future context in the sequence while the attention mechanism helps with relevant keywords in the sentence. CNN-BiGRU was used as CNN helps to capture the local patterns and keywords like “very good” while the BiGRU part was used to capture sequential dependencies in both directions. The transformer model has self-attention layers which captures relationships for all words in the context. It handles long range depencies well and unlike LSTM and BiGRU it has positional encoding which allows the model to understand the order of words.

B. Parameter Selection

The parameters selected for the BiLSTM and CNN-BiGRU are acheieved through hyperparameter tuning to find the best accurracy. For the BiLSTM the size of the hidden layers and how many stack layers is experimented to get the best accuracy and lowest loss. For the CNN-BiGRU model, the CNN layer was tested with various filter and kernel size and the configuration pooling layer. For the BiGRU layer, it is similar to the BiLSTM layer where different hidden layer sizes were tested and the number of stacked layers to achieve the best accuracy. The dropout rate of these models were tuned based on the model performance on the train and validation data to prevent overfitting of the model. For the transformer model the number of layers and number of heads were experimented with and tested to get the best results. Multiple attention mechanism was tested to find the best attention mechanism to be used for the transformer model. The optimizer was found from testing various optimizers and Adam was found to be the most effective with the best accuracy. The learning rate of the models are tuned based on the convergence of the model and its accuracy.

C. Evaluation Methods

The evaluation methods used are the accuracy, F1 Score, Confusion Matrix and ROC of the test data (unseen data). The accuracy is the measurement of the percentage of correct predictions for the test set. The F1 Score is the measurement of the harmonic mean of Precision and Recall scores which measures a balanced result for Precision and Recall which shows the true positives and actual positives. The confusion matrix will give a detailed visualisation of the models predictions against the actual labels. It tells the True Positives, False Negatives, False Positives and True Negatives for the prediction. Finally the Receiver Operating Characteristic (ROC) Curve and the Area Under the Curve (AUC). This graph is a plot of the true positive rate over the false positive rate. The AUC for each class tells how well the model predicts and seperates each class. A value closer to 1 is better and 0.5 would mean that the model is random guessing.

D. Dataset

The dataset used for this experiment is the wassa dataset which is a collection of twitter data with labeled emotion of each text. The emotions category consists of anger, fear, joy and sadness. It also has an intensity label from 0 to 1 to show the intensity of the emotion from the text. The data from the wassa is split into training validation and testing with a total of 7102 data and 19117 vocabulary size.

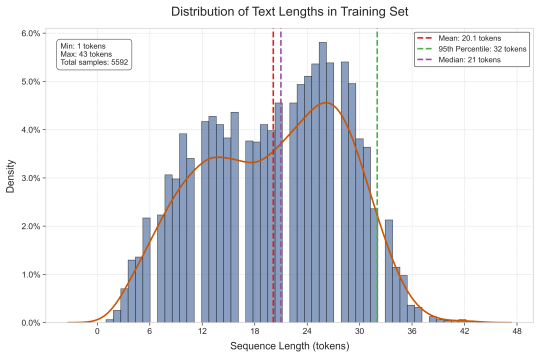

Fig. 11. Token Length Distribution in Wassa Dataset, the maximum number of tokens in a sentence is 43 tokens.

The graph shows the distribution of the data sequence length for the training dataset. We can see that the bulk of the data lies between 12 – 30 sequences. The table below shows the sequence length statistics for the training, validation and test datasets. Knowling the sequence length would help us identify the ideal max sequence length for the dataloader.

	Training	Validation	Test
Min length	1	3	2
Max length	43	46	48
Mean length	20	20	20
Median length	21	21	20
95th percentile	32	32	32
99th percentile	35	35	34

TABLE I
SEQUENCE LENGTH STATISTICS FOR DATASET

IV. Experiments and Results

A. Attention-based Bidirectional Long Short-Term Memory (BiLSTM)

To evaluate the effectiveness of attention mechanisms in sequence modeling for multi-class emotion classification, we conducted a comparative analysis between a standard Bidirectional Long Short-Term Memory (BiLSTM) model and an enhanced Attention-based BiLSTM architecture.

1) BiLSTM without Attention: The baseline BiLSTM model, while capable of learning contextual dependencies, demonstrated limited performance on both the validation and test sets. These results indicate that, in the absence of an attention mechanism, the model struggled to effectively differentiate between emotion classes, likely due to its inability to prioritize the most informative parts of the input sequence.

Metric	Score
Best Validation F1 Score	0.1481
Test Accuracy	0.3319
Test F1 Score	0.1654

TABLE II
PERFORMANCE METRICS FOR BASELINE BILSTM MODEL

2) Proposed Model: Attention-Based BiLSTM: Integrating an attention mechanism significantly improved the model's capacity to capture salient features relevant to emotion classification. The attention-enhanced BiLSTM achieved markedly better performance.

Metric	Score
Best Validation F1 Score	0.8680
Test Accuracy	0.8650
Test F1 Score	0.8657

TABLE III
PERFORMANCE METRICS FOR ATTENTION-BASED BILSTM MODEL

This substantial performance gain underscores the critical role of attention in enhancing the representational power of sequence models by enabling a more focused and interpretable analysis of textual inputs. The emotions are represented by Class 0 (Anger), Class 1 (Fear), Class 2 (Joy), Class 3 (Sadness) for the Confusion matrix and the Receiver Operating Characteristic (ROC). ROC curve analysis, visualized in Figure 13, further validated the superiority of the attention-based model. The Area Under the Curve (AUC) scores for each class were consistently high.

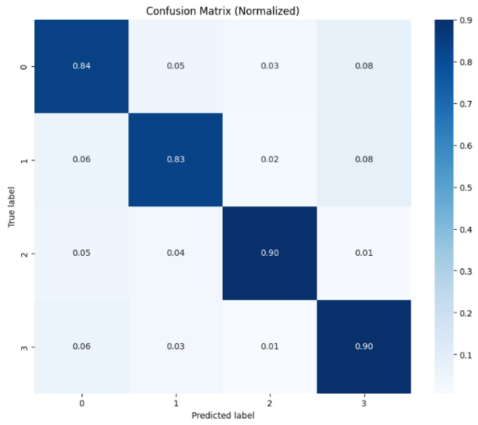

Fig. 12. Confusion Matrix for BiLSTM

These findings demonstrate that the attention mechanism not only improves overall accuracy and F1 score but also enhances the model's ability to discriminate between fine-grained emotion classes. These results collectively demonstrate the efficacy of the Attention-based BiLSTM model in emotion classification tasks, affirming its suitability for practical deployment in text emotion recognition and related applications.

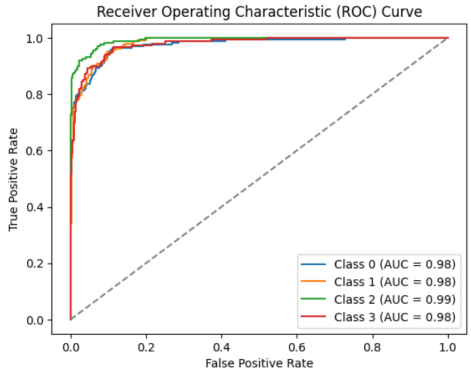

Fig. 13. ROC-AUC for BiLSTM

B. Convolutional Neural Network with Bidirectional Gated Recurrent Unit (CNN-BiGRU)

This hybrid architecture leverages the strengths of both convolutional and recurrent components: convolutional layers serve to extract local features such as n-gram patterns, while the BiGRU layers model long-range contextual dependencies in both forward and backward directions. The emotions are represented by Class 0 (Anger), Class 1 (Fear), Class 2 (Joy), Class 3 (Sadness) for the Confusion matrix and the Receiver Operating Characteristic (ROC).

1) BiGRU without CNN: The baseline BiGRU model was tested without the CNN layer to see the performance.

Metric	Score
Best Validation F1 Score	0.1188
Test Accuracy	0.3319
Test F1 Score	0.1246

TABLE IV
PERFORMANCE METRICS FOR BASELINE BIGRU MODEL

The BiGRU model also struggled to capture the representation for each class which may be due to not having enough patterns for the BiGRU to work with.

2) CNN-BiGRU: On the unseen test dataset, the CNN-BiGRU achieved the following results.

Metric	Score
Best Validation F1 Score	0.8790
Test Accuracy	0.8833
Test F1 Score	0.8855

TABLE V
PERFORMANCE METRICS FOR CNN-BIGRU MODEL

The addition of the CNN layer boosts the performance of the BiGRU model significantly. The CNN layer acts as a form of preprocessing for before the BiGRU layer by capturing the

local features making it easier for the BiGRU model to process afterwards.

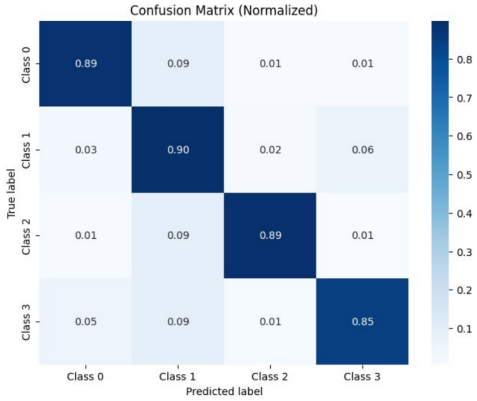

Fig. 14. Confusion Matrix for CNN-BiGRU

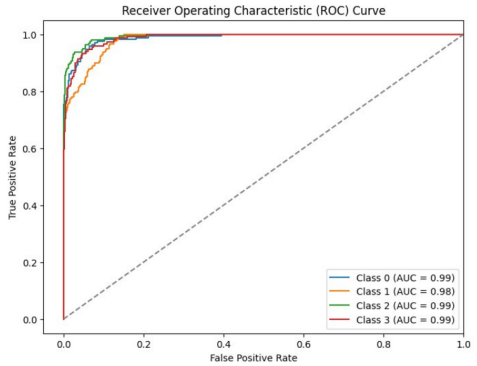

Fig. 15. ROC-AUC for CNN-BiGRU

These results indicate that the model effectively captures both the structural and sequential nuances of emotional language, leading to a more robust and nuanced classification performance. The superior F1 score reflects a strong balance between precision and recall across all emotion classes, suggesting consistent performance in identifying the full range of emotional expressions. The success of this model highlights the complementary roles of convolutional and recurrent processing. While CNN layers specialize in identifying localized textual features (e.g., emotional phrases or key expressions), the BiGRU layers enhance sequence understanding by modeling dependencies across the entire input. Given its high accuracy and F1 score, the CNN-BiGRU model presents itself as a highly effective approach for emotion classification tasks in real-world NLP applications.

C. Transformer-based Models

In here, we experimented on multiple attention architectures Multi-Head Attention (MHA), Multi-Query Attention (MQA), Grouped-Query Attention (GQA) and Multi-Head Latent Attention (MLA). Table VI summarizes the key performance metrics for each transformer variant:

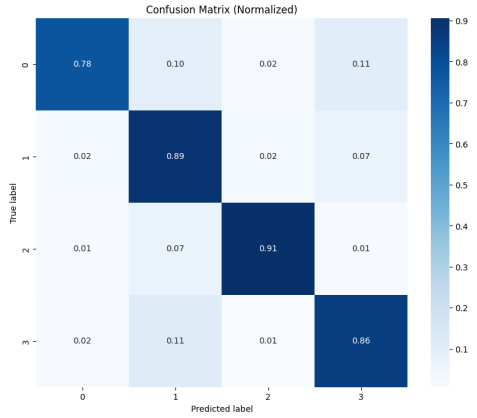

Fig. 16. Confusion Matrix for Transformer_GQA

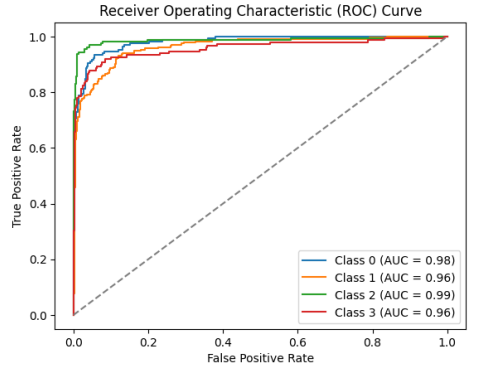

Fig. 17. ROC-AUC for Transformer_GQA

Our comprehensive evaluation of transformer-based models with different attention mechanisms reveals significant insights into the trade-offs between performance, computational efficiency, and model complexity in emotion classification tasks.

The Grouped-Query Attention (GQA) model achieved the highest test F1 score of 0.8649, marginally outperforming the traditional Multi-Head Attention (MHA) baseline which scored 0.8625. This suggests that GQA's approach of sharing key and value projections across query groups effectively balances information sharing and specialization. The confusion matrix for the GQA model demonstrates strong diagonal dominance, indicating robust classification performance across all emotion categories. The ROC curves further validate GQA's effectiveness, with high Area Under Curve (AUC) values for all emotion classes, demonstrating excellent discrimination capability even for challenging emotion distinctions.

V. Discussion

A. BiLSTM and CNN-BiGRU

The comparative analysis of the BiLSTM-based models and the CNN-BiGRU architecture reveals important insights into the effectiveness of different neural architectures for multi-class emotion classification.

TABLE VI
PERFORMANCE COMPARISON OF TRANSFORMER-BASED MODELS

Model	Test F1	Val F1	Training Time (ms)	Attention Params (M)	Inference Time (ms)
MHA	0.8625	0.8727	20.91	0.24	7.15
MQA	0.8599	0.8611	20.50	0.13	6.56
GQA	0.8649	0.8721	24.46	0.15	7.38
MLA	0.8377	0.8453	18.82	0.15	6.37

1) BiLSTM Models: Limitations and Improvements through Attention: The baseline BiLSTM model, which lacks an attention mechanism, struggled to capture meaningful patterns in emotional language. Its low performance (Test F1 Score: 0.1654) suggests that while BiLSTM layers are theoretically capable of modeling bidirectional context, in practice, they may be insufficient for distinguishing subtle emotional cues without additional guidance. On the other hand, the integration of an attention mechanism improved the model's capability remarkably. The Attention-based BiLSTM showed a significant jump in performance (Test F1 Score: 0.8657), indicating that attention enables the model to focus on emotionally salient tokens within the input sequence. This targeted focus enhances interpretability and effectiveness, especially in complex or nuanced emotional expressions. It also suggests that long-range dependency modeling, when combined with selective emphasis on key features, is critical in understanding emotional language. Despite these gains, the attention-based BiLSTM still has limitations. While it performs well, it may be less efficient in learning local features (e.g., fixed phrase patterns), which are often crucial for emotion recognition. Furthermore, the LSTM-based architecture tends to be computationally heavier and slower to train compared to GRU-based or convolutional alternatives.

2) CNN-BiGRU: A Stronger Hybrid Approach: The baseline BiGRU model struggled to capture the pattern for this dataset with a low F1 score and accuracy of 0.1188 and 0.3319, respectively. The model is designed to predict 4 classes where an accuracy of 0.3319 is very close to random guessing of the classifications. By adding the CNN layer, the CNN-BiGRU architecture outperformed both BiLSTM variants, achieving the highest test accuracy (0.8833) and F1 score (0.8855). This success can be attributed to the hybrid design that effectively combines local and global feature learning. CNN layers are adept at extracting local patterns, such as emotionally charged phrases or idiomatic expressions, while BiGRU layers efficiently model sequential dependencies with fewer parameters and faster convergence than LSTM-based units. This synergy between CNN and BiGRU allows the model to generalize well across varied expressions of emotion, whether explicit or implicit. Moreover, the GRU's simplified gating mechanism offers a more computationally efficient alternative to LSTM, which is beneficial for training on large datasets or deploying in real-time applications.

3) Comparative Insights: Both attention mechanisms and convolutional layers offer distinct advantages in sequence modeling, yet their contributions address different aspects of the emotion classification task. Attention mechanisms enhance the model's ability to selectively focus on emotionally salient words, improving interpretability and performance in context-rich sequences. In contrast, convolutional layers are highly effective at detecting local patterns—such as specific emotional phrases or word combinations—that often serve as strong indicators of sentiment. The CNN-BiGRU architecture, by integrating both local pattern extraction and temporal sequence modeling, strikes an optimal balance between performance and computational efficiency. Its superior results underscore the importance of capturing both granular textual features and broader contextual relationships for robust emotion recognition. The synergy between convolutional and recurrent components enables the model to generalize across diverse emotional expressions with greater precision. These findings emphasize that model architecture should be closely aligned with the characteristics of the data and the complexity of the task. Emotion classification spans a wide spectrum from brief and explicit cues to subtle and context-dependent expressions. As such, models that incorporate multi-level representation learning, combining local feature extraction with global sequence understanding, are better equipped to handle this variability. This suggests a promising direction for future research on the development of hybrid architectures that can adaptively focus on both fine-grained and long-range emotional

B. Transformer-based Models

Our experimental results demonstrate that transformer-based architectures performed on par with traditional approaches for text emotion recognition, possibly due to the small dataset size. Among the transformer variants we tested, each exhibits distinct performance characteristics that offer important insights for model selection in practical applications.

Some of our key findings:

1) GQA demonstrates superior performance (highest F1 scores) with only 0.15M attention parameters, confirming the theoretical advantage of grouped attention's middle-ground approach between full specialization (MHA) and complete sharing (MQA).
2) MHA has significantly higher attention parameters (0.24M) yet achieves lower F1 scores than GQA, suggesting that full independent attention heads may lead to redundancy without proportional performance gains.
3) MQA shows good efficiency with 46% fewer attention parameters than MHA while maintaining competitive F1 scores, validating the hypothesis that query sharing can reduce computational requirements with minimal performance impact.
4) MLA (Multi-Head Latent Attention) achieves the fastest training and inference times but the lowest F1 scores, due to its simplified attention mechanism that uses dimension reduction – latent representation in the attention computation, reducing the computational complexity of the self-attention operation.

Among the transformer variants, GQA strikes the best balance between performance and computational efficiency. The superior performance of GQA can be attributed to its ability to maintain sufficient specialization of attention heads while reducing parameter redundancy. By grouping query heads to share key-value pairs, GQA enables more efficient memory usage during inference, reduced computational requirements compared to MHA, and increased specialization compared to MQA. MHA stores separate key-value pairs for each attention head, resulting in the largest memory footprint but enabling maximum representational capacity. In contrast, MQA dramatically reduces memory requirements by sharing key-value pairs across all heads, but this constraint impacts performance. Meanwhile, MLA uses a dimensionality reduction approach, projecting attention computation into a lower-dimensional latent space to reduce memory usage while maintaining competitive performance.

These findings align with recent research in transformer optimization, suggesting that grouped attention mechanisms offer a promising direction for improving both performance and efficiency in emotion recognition tasks.

TABLE VII
COMPARISON OF DIFFERENT ATTENTION MECHANISMS

Attention Mechanism	**KV Cache Entries Per Token**	**KV Cache Size Per Token***	**Performance**
Multi-Head Attention (MHA)	$2n_h d_h l$	4 MB	High
Multi-Query Attention (MQA)	$2d_h l$	31 KB	Lower
Grouped-Query Attention (GQA)	$2n_g d_h l$	500 KB**	Medium
Multi-Head Latent Attention (MLA)	$d_l l$	70 KB	Higher

Note: This table provides a comparison of memory requirements and performance characteristics across different attention mechanisms. n_h represents the number of attention heads, d_h is the dimension per head, l is the sequence length, n_g is the number of groups in GQA, and d_l is the latent dimension in MLA. * KV Cache Size Per Token is calculated assuming 16-bit floating point (FP16) representation with $n_h = 32$, $d_h = 64$, $n_g = 8$, and $d_l = 8$. ** GQA's memory footprint scales with the number of groups (n_g), providing a configurable trade-off between MHA and MQA.

C. Summary View

Contrary to theoretical expectations, our experiments demonstrate that CNN-BiGRU outperformed transformer-based models in text emotion recognition tasks, achieving the highest F1 score of 0.8855. This hybrid architecture effectively combines CNN's ability to extract local emotional features with BiGRU's capacity to model sequential dependencies, creating a powerful framework for emotion classification that exceeded both the Attention-BiLSTM (0.8657) and all transformer variants.

Among the transformer models evaluated, Grouped-Query Attention (GQA) emerged as the most effective approach, achieving the highest test F1 score (0.8649) within the transformer family while maintaining moderate computational overhead. GQA's architecture strikes a compelling balance between expressiveness and parameter efficiency by reducing redundancy in the attention computation, making it well-suited for practical deployment in emotion classification tasks that demand both reasonable precision and real-time inference capabilities.

The comparison of attention variants highlights how even subtle architectural modifications in attention design, such as sharing query-key weights (as in MQA) or compressing latent representations (as in MLA), can lead to meaningful trade-offs in performance, training time, and resource consumption. These findings reinforce the notion that attention mechanism design is not merely a detail but a significant determinant of model effectiveness, especially in tasks involving nuanced emotional cues spread across varying text lengths and structures. However, our results also emphasize that traditional hybrid architectures like CNN-BiGRU remain highly competitive and should not be overlooked in favor of newer transformer-based approaches for specialized NLP tasks such as emotion recognition.

D. Limitations and Future Work

While our results demonstrate the effectiveness of transformer-based models, particularly GQA, for text emotion recognition, several limitations should be noted. The relatively modest dataset size may influence the relative performance of the models, potentially favoring architectures that generalize well from limited examples. Additionally, computational constraints limited our exploration of deeper transformer architectures that might yield further improvements in classification accuracy. The current study also focuses exclusively on English text, leaving multilingual emotion recognition as a significant challenge that requires additional investigation. These limitations suggest caution when generalizing our findings to more diverse linguistic contexts or larger-scale deployments.

The results from the experiments open up promising directions for further exploration. Future research may benefit from investigating adaptive or hybrid attention mechanisms that dynamically adjust based on sequence length, emotional complexity, or input ambiguity. Additionally, incorporating context-aware or hierarchical attention—potentially informed by syntactic or semantic structure—could further enhance the model's ability to capture layered emotional signals within text. Such innovations would move us closer to more human-like understanding of emotional nuance in natural language processing systems.

Other hybrid models like Transformers + CNN + BiLSTM/BiGRU could be explored with CNN capturing local features, BiLSTMs capturing temporal dependencies and the transformers capturing long-range dependencies. The behaviour of this model could be investigated with different

datasets with different sequence length to understand its performance and adaptability.

Multilingual datasets could also be used to research on the performance of how transformer models and BiLSTM or BiGRU models perform and if the knowledge of training on one language could be transferred to another language by fine tuning the models. Zero shot learning or few shot learning can also be tested for these models where another language is used for the testing instead to see how well the model is able to adapt to different languages.

VI. Conclusion

This study provides a comprehensive comparison of deep learning approaches for text emotion recognition, from established BiLSTM models to state-of-the-art transformer architectures. Our results demonstrate that hybrid architectures combining convolutional and recurrent components achieve the best performance for this specific task.

The CNN-BiGRU model achieves the highest results with an F1 score of 0.8855, outperforming both the Attention-based BiLSTM (0.8657) and the best transformer variant, GQA (0.8649). This suggests that the combination of CNN's local feature extraction capabilities with BiGRU's sequential processing offers particular advantages for emotion recognition tasks.

Among transformer architectures, the GQA model demonstrates the best performance within its family, highlighting the effectiveness of grouped-query attention in balancing computational efficiency with modeling capability. Although transformer models did not surpass the hybrid CNN-BiGRU in our experiments, they remain competitive and offer valuable architectural insights, particularly regarding attention mechanism design.

These findings contribute to the ongoing development of more accurate and efficient emotion recognition systems, with potential applications in sentiment analysis, customer feedback processing, mental health monitoring, and human-computer interaction. Our research emphasizes that model selection should be guided by empirical evaluation rather than theoretical novelty, as established architectures may still offer superior performance for specialized NLP tasks.

References

[1] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global Vectors for Word Representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1532–1543.

[2] Stanford NLP Group, "GloVe: Global Vectors for Word Representation," [Online]. Available: https://nlp.stanford.edu/projects/glove/

[3] A. Vaswani et al., "Attention is All You Need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.

[4] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," in *International Conference on Learning Representations*, 2015.

[5] P. Zhou et al., "Attention-Based Bidirectional Long Short-Term Memory Networks for Relation Classification," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016, pp. 207–212.

[6] A. K. Roy, "Emotion Classification in Social Media using BERT and Bi-LSTM with Attention," *American Journal of Science and Technology*, vol. 5, no. 3, 2023, doi: 10.54097/ajst.v5i3.7347.

[7] Y. Wang et al., "A CNN-BiGRU Text Emotion Recognition Model Fusing Character and Part-of-Speech Features," in *IEEE Access*, vol. 8, pp. 90729–90740, 2020.

[8] M. Wang, L. He, Y. Wang, and H. Liu, "Emotion Recognition Based on CNN-LSTM Fusion Feature from EEG Signals," *IEEE Access*, vol. 7, pp. 8804–8813, 2019.

[9] J. Ainslie, J. Lee-Thorp, B. De Jong, J. Dao, S. Wheadon, B. Chandra, and S. Fedus, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," arXiv preprint arXiv:2305.13245, 2023.

[10] N. Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need," arXiv preprint arXiv:1911.02150, 2019.

[11] T. Lee and Z. Yang, "Latent Attention for Linear Time Transformers," in *International Conference on Learning Representations*, 2023.

[12] R. Alhamid et al., "Personalized Emotion-Aware Healthcare Recommendation: An Ensemble Learning Perspective," *IEEE Access*, vol. 6, pp. 40362–40374, 2018.

[13] M. C. O. Ferrera, M. Kamel, and S. Di Gennaro, "An Explainable Deep Learning Approach for Emotion Classification," *Journal of AI and Data Mining*, vol. 5, no. 3, 2023.

[14] X. Zhang, Y. Wang, C. Liu, J. Yu, and D. Li, "Research on CNN-based Emotion Recognition Algorithm," in *Proc. 15th IEEE Int. Conf. Electronic Measurement & Instruments (ICEMI)*, 2018.

[15] I. K. Ihianle, A. O. Nwajana, S. H. Ebenuwa, R. I. Otuka, K. Owa, and M. O. Orisatoki, "A Deep Learning Approach for Human Activities Recognition from Multimodal Sensing Devices," *IEEE Access*, vol. 8, pp. 179028–179038, 2020.

[16] X. Zhao, L. Wang, Y. Zhang et al., "A Review of Convolutional Neural Networks in Computer Vision," *Artificial Intelligence Review*, vol. 57, p. 99, 2024.

[17] A. Zhang, Z. Lipton, M. Li, and A. Smola, "Modern Recurrent Neural Networks," in *Dive into Deep Learning*, 2023. [Online]. Available: https://d2l.ai/chapter_recurrent-modern/lstm.html

[18] A. Zhang, Z. Lipton, M. Li, and A. Smola, "Gated Recurrent Units (GRUs)," in *Dive into Deep Learning*, 2023. [Online]. Available: https://d2l.ai/chapter_recurrent-modern/gru.html

[19] O. Calzone, "An Intuitive Explanation of LSTM," *Medium*, 2020.

[20] A. Nama, "Understanding Bidirectional LSTM for Sequential Data Processing," *Medium*, 2020.

[21] D. Mol, "Attention Mechanism in Deep Learning," *Deep Learning Book Companion*, 2021.

[22] IBM, "What is an Attention Mechanism in AI?," *IBM Think Blog*, 2021.

[23] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[24] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[25] I. Goodfellow, Y. Bengio, and A. Courville, "Sequence Modeling: Recurrent and Recursive Nets," in *Deep Learning*, MIT Press, 2016, ch. 10, pp. 367–415.

[26] N. Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need," arXiv preprint arXiv:1911.02150, 2019.

[27] J. Ainslie, J. Lee-Thorp, B. De Jong, J. Dao, S. Wheadon, B. Chandra, and W. Fedus, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," arXiv preprint arXiv:2305.13245, 2023.

[28] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling Laws for Neural Language Models," arXiv preprint arXiv:2001.08361, 2020.

[29] F. Meng, Z. Yao, and M. Zhang, "TransMLA: Multi-Head Latent Attention Is All You Need," arXiv preprint arXiv:2502.07864, 2025.